

Diagnostic Structurel IA — Project Guide

Who this is for: anyone joining the project who hasn't followed the BMad planning process. Read this first — it explains what the product is, how the planning method works, which documents exist, in what order to read them, and where we are before coding starts.

Planning documents are written in **English**; the **product itself ships in French** (hard requirement).

1. What the product is

Diagnostic Structurel IA is a mobile-first, offline-capable field tool for French structural-diagnostic experts, plus a web back-office.

The core insight: a full structural diagnostic takes **~3.5 days, but only ~0.5 day is on-site**. The rest is deskwork — writing the report, placing annotated photos, drawing technical *coupes* and *schémas*. This product attacks the deskwork, not the inspection:

- **On site, offline:** guided survey (*Relevé*) by structure type — checklists, 1–5 intensity sliders, geolocated photos pinned to structural elements. No network needed, zero data loss.
- **AI-assisted reading:** synced photos are pre-classified by disorder type (*fissuration, corrosion, humidité, déformations, maçonnerie, divers*) with severity and a confidence score. The AI also drafts cause hypotheses and recommendations as editable text.
- **Compliant PDF deliverable, generated:** cover page, TOC, per-zone synthesis, annotated photos, *coupes*, *schémas*, and disorder mapping — assembled automatically.
- **Learning loop:** every expert correction becomes training data for periodic model retraining.

The one non-negotiable principle: *the AI proposes, the Expert disposes and signs*. French law requires a qualified engineer to sign every diagnosis and carry professional liability — the AI never diagnoses autonomously. Every AI output is validated, corrected, or rejected by the Expert before it appears in any report.

Why now: Décret n°2025-814 (Aug 2025, *loi Habitat dégradé*) makes structural diagnosis of collective housing mandatory in designated zones — rising demand against an unchanged manual bottleneck.

Who it's for (v1): the two founding experts themselves — it's a POC that must prove itself on their real missions before any commercial rollout.

2. How the BMad workflow works

BMad (Breakthrough Method for Agile AI-Driven Development) is a structured planning method where AI agents and humans co-produce a chain of documents, each one feeding the next. The point is that **by the**

time coding starts, an AI developer agent has an unambiguous, self-contained contract for every piece of work.

The chain used in this project:



Two things worth understanding about the method:

- **Adversarial reviews.** Most documents were deliberately attacked by review passes (cynical reviews, seam analysis, currency verification) and then reconciled. The `review-*.md` and `reconcile-*.md` files are the audit trail of that process — you don't need to read them, but they explain *why* the main documents say what they say.
- **Cross-referencing.** Documents cite each other precisely: stories cite FRs (`FR-11`), FRs map to capabilities (`CAP-9`), architecture rules are numbered (`AD-4`), acceptance criteria cite the Cahier (`§9.1.b`). When you see one of these codes, it's a pointer into another document in the chain.

3. The documents — reading order

Everything the workflow produced lives in `_bmad-output/`. Here is the recommended reading order, with what each document gives you.

Step 1 — Understand the product (30 min)

Read	What you get
Product Brief	The best single overview: problem, solution, differentiation, competition, success criteria. Start here.
Brief Addendum	Research depth behind the brief: market, regulatory digest, and the preserved §9 acceptance criteria from the original Cahier.

Step 2 — Understand the requirements (1–2 h)

Read	What you get
PRD	The scope contract for v1: vision, user journeys (UJ-1...3), the glossary (§4 — the project's vocabulary, use these terms exactly) , all 26 functional requirements, NFRs, open questions, assumptions.
PRD Addendum	Competitive landscape (T2D2, Spectora, PlanRadar...), full disorder taxonomy, technical depth that doesn't belong in the PRD narrative.
SPEC	The canonical machine contract: 14 capabilities (CAP-1...CAP-14) with intent + success conditions. Downstream work binds to these.
Acceptance Criteria	The test contract — the French §9 criteria the pilot experts will validate against, mapped to capabilities with measurable thresholds.
Compliance References	The regulatory backdrop (décret, signer qualification, liability) driving the "AI never signs" constraint.

Step 3 — Understand how it will be built (1 h)

Read	What you get
Architecture Spine	The build substrate: local-first with server-authoritative reconciliation , ports-and-adapters, and the numbered invariants (AD-1...) that all code must respect — e.g. UI only ever reads/writes local SQLite, photos never travel in the sync stream, one single PDF renderer compiled into both app and server.
Project Context	The distilled rulebook for AI coding agents: exact tech stack and versions, package-purity rules, naming conventions (French domain nouns in code: <code>Desordre</code> , <code>Releve</code> , <code>Coupe</code> ...), error-model rules. Any agent (or human) writing code must follow this.

Step 4 — Understand the work plan (30 min)

Read	What you get
Epics & Stories	The full breakdown: requirements inventory, then 6 epics / 30 stories , each with acceptance criteria. This is the implementation backlog.

The six epics:

1. **Project Foundation, Missions & Access** — scaffold, auth, roles, mission creation, expert assignment
2. **Offline Guided Field Survey** — dynamic forms, rapid entry, photos, annotation, zero-loss sync
3. **Coupes, Schémas & Désordre Mapping** — coupe library, disorder positioning, sketch tool
4. **Compliant Rapport Generation & Sharing** — PDF document model, templates, offline render, export, provenance
5. **AI Detection & Expert Validation** — analysis pipeline, model registry, validation loop
6. **Learning Loop, Dashboard & Pilot Readiness** — training-data capture, dataset export, AI dashboard, documentation

Reference / audit trail (read only if needed)

Document	Purpose
Cahier des Charges	The founders' original requirements (French, v1.1, May 2026 — §1–3 only; its §9 acceptance criteria survive in the brief addendum and the spec's acceptance-criteria file). Feature-detail authority.
PRD reviews — validation report, adversarial review, rubric, reconcile-brief, reconcile-cahier	Proof the PRD was attacked and reconciled against its sources.
Architecture reviews (reviews/)	Seam analysis and currency verification of the spine.
bmad-user-guide.md	Generic BMad method manual (how the agents/skills work).

4. Key decisions already locked in

So you don't re-litigate them:

- **Tech stack:** Flutter/Dart for mobile (iOS + Android) and web back-office; SQLite on device + PowerSync + Supabase (Postgres, EU region — **EU data residency is an invariant**); pure-Dart PDF renderer; Python 3.12 + FastAPI + PyTorch inference service; Claude API for text drafting behind a swappable port.
- **Offline-first is a hard constraint**, not a feature: a full survey with photos must work with zero network and sync losslessly within 30 s of reconnection.
- **AI accuracy commitment is honest and narrow:** >75% precision on cracks and corrosion only, always expert-validated; other categories best-effort. Severity proposals carry no accuracy commitment in v1.
- **French domain vocabulary is kept verbatim** in documents *and* in code (`Desordre` , `ReLeve` , `Coupe` , `Ouvrage` — never translated).
- **No unvalidated AI content ever reaches a report**, and every report keeps an immutable provenance record of what was AI-drafted vs. expert-authored.

5. Repository layout

— README.md	← you are here
— bmad-user-guide.md	← generic BMad method manual
— docs/	← original Cahier des Charges
— _bmad-output/	← everything the planning workflow produced
— project-context.md	← rulebook for AI coding agents
— planning-artifacts/	
— briefs/...	← product brief + addendum
— prds/...	← PRD + addendum + reviews
— architecture/...	← architecture spine + reviews
└ epics.md	← 6 epics / 30 stories
└ specs/	
└ spec-diagnostic-structurel-ia/	← SPEC + acceptance criteria + compliance
— _bmad/	← BMad framework configuration (not project content)
└ .claude/skills/	← the BMad agent skills used to run the workflow (tooling)

Rule of thumb: `_bmad-output/` is the project; `_bmad/` and `.claude/` are the machinery that produced it — you never need to read the machinery.

6. Where we are, and what happens next

Status: planning is complete; no application code exists yet.

The next BMad phases, in order:

1. **Sprint planning** — generate a sprint status/tracking file from the epics.
2. **Create story** — for each story, produce a dedicated story file packing in all the context (requirements, architecture rules, acceptance criteria) an AI developer agent needs.
3. **Dev story** — the developer agent implements the story, respecting `project-context.md` and the architecture spine.
4. **Code review & retrospective** — adversarial review of the code, then lessons after each epic.

Open items worth knowing before coding (details in PRD §10 and the spine):

- The **official arrêté report template** is not yet published — FR-19 carries the contingency; the template system must absorb it when it lands.
- The **detection model architecture is UNDECIDED** (must be Apache/BSD/MIT-licensed — no AGPL in the deployed path).
- The **AI acceptance-testing gate** requires ≥ 150 usable labeled examples per disorder category; fallback v1 scope is cracks + corrosion only.

7. Quick FAQ

Do I need to install or learn BMad to contribute? No. BMad was used to *produce* the documents; reading them requires nothing. If you want to run the workflow yourself (e.g. create the next story), it runs through Claude Code skills — see [bmad-user-guide.md](#).

Which document wins if two disagree? The SPEC (+ its companions) is the canonical contract for *what* to build; the Architecture Spine for *how*; the PRD for scope questions; the brief for intent/positioning. The epics file lists its exact input documents in its header.

Why are the documents in English if the product is French? Deliberate project rule (NFR-3): planning in English, product UI / forms / AI text / reports entirely in French. French domain terms stay French everywhere because they're the shipped vocabulary.